

Relatório Técnico — Investigação das APIs Pandapé v2 e v3

Existe endpoint de listagem/descoberta de candidatos que a v1 não tinha? Impacto na arquitetura da Fase 5 (cron-pull × webhook)

Projeto: EA AUTOMATIC — Esteira Admissional (Grupo Soulan) **Integração:** INT-1 (Pandapé / ATS) **Data dos testes:** 1º de julho de 2026 **Autor técnico:** Fábrica EA AUTOMATIC (coordenação de engenharia) **Destino:** Diretoria + Time de Infraestrutura — reavaliação da decisão cron-pull × webhook **Classificação:** Interno — **sem dados pessoais de candidato** (apenas estrutura técnica; ver nota §2.1) **Documento par:** docs/RELATORIO-INVESTIGACAO-API-PANDAPE.md (investigação da API v1, 2026-06-30)

1. Sumário executivo — resposta direta

Pergunta: as versões v2 e v3 da API do Pandapé têm algum endpoint de **listagem ou descoberta de pré-colaboradores/candidatos novos** (PreCollaborator/List, /Search, ou filtro por data/"mudanças desde") que **não existia na v1**?

A resposta tem **duas camadas** — e é importante não confundir uma com a outra:

1) Sobre o recurso PreCollaborator especificamente — NÃO mudou nada. Em **v1, v2 e v3**, PreCollaborator continua sendo um único endpoint *get-by-id* (GET /vN/precollaborators/{idPreCollaborator}). **Não há PreCollaborator/List, /Search, nem filtro "changes-since" em nenhuma versão.** Confirmado ao vivo (v2 e v3 → HTTP 404 em id inexistente).

2) MAS a v2 introduz uma capacidade de enumeração de candidatos que a v1 NÃO tinha — e ela é real. O módulo Match ganhou, na v2, um endpoint de **listagem paginada filtrável por vaga e etapa**: GET /v2/matches?IdVacancy=&IdVacancyFolder=&Page=&PageSize=. Na v1, Match era só *get-by-id*. **Testado ao vivo com a credencial real: HTTP 200, retornando 253 candidatos para uma única vaga**, e cada registro traz **cpf, idVacancyFolder (etapa) e modifyDate/insertDate (carimbos de tempo)** — ou seja, é filtrável por etapa e permite *delta* do lado do cliente.

Conclusão honesta: isto reabre a questão de arquitetura, mas **NÃO encerra o webhook por si só**. Existe agora um caminho plausível de *pull* para **descobrir candidatos** (vaga → etapa → matches) que não existia antes. Porém, para **substituir o webhook na Fase 5**, esse caminho esbarra em **quatro lacunas concretas** (detalhadas na §6) que precisam ser resolvidas antes de qualquer decisão:

- O registro de Match **não expõe idPreCollaborator** — e os **documentos** (insumo da auditoria F2) só existem no objeto PreCollaborator. A ligação match→pré-colaborador **não está exposta**.
- Não há filtro "mudanças desde" no servidor** — o *delta* seria feito no cliente por modifyDate.
- Fan-out pesado:** há **6.821 vagas** na conta; varrer *vaga×etapa* a cada ciclo do cron precisa de uma estratégia de escopo sob o teto de **1.000 req/5min** compartilhado (risco à folha, §A.5).
- A "etapa de admissão" é uma **pasta de nome livre por vaga** (ex.: Contratados) — exige **de/para**.

Recomendação: tratar como **descoberta promissora que exige uma prova de conceito (spike) de viabilidade**, não como troca imediata de arquitetura. E, conforme pedido do próprio Pandapé, **qualquer novo endpoint deve ser alinhado com o time de Operações deles antes de ir a produção (§7)**. **Nada foi implementado** — este documento é apenas o resultado da investigação.

2. Ambiente e metodologia

2.1 Ambiente

Item	Valor
Data	2026-07-01
API base	https://api.pandape.com.br (produção)
Autorização	https://login.pandape.com.br/connect/token (OAuth2 client_credentials, escopo PandapeApi)

Credencial	A MESMA já configurada (PANDAPE_CLIENT_ID/PANDAPE_CLIENT_SECRET em apps/backend/.env)
Specs de referência	.../swagger/v2/swagger.json (261 KB) e .../swagger/v3/swagger.json (28 KB) — OpenAPI 3.0.1
Autenticação (resultado)	Sucesso — Bearer emitido; probes de leitura executados ao vivo

Nota de segurança (LGPD §A.6): os *probes* ao vivo tocaram endpoints que retornam dados pessoais (o listar-matches inclui cpf, nome, e-mail, telefone). **Nenhum valor de candidato foi exibido, persistido ou logado** — as evidências deste relatório contêm apenas **códigos HTTP**, **contagens agregadas (totalItems)** e **NOMES de campos** (nunca valores). `client_secret` e `access_token` nunca foram exibidos. Identificadores estruturais não-pessoais (ex.: `idVacancy=847`, nomes de etapas como `Contratados`) são metadados de configuração, não PII.

2.2 Método

1. **Download das specs OpenAPI oficiais** das duas versões, direto da API.
2. **Enumeração completa** dos endpoints de v2 (68 paths / 86 operações) e v3 (4 paths / 5 operações), por módulo.
3. **Varredura dirigida ao discovery:** busca, em *todos* os endpoints, por qualquer operação de listagem/busca de candidatos ou por parâmetros de `since/■date/■updated/■changed/■page`.
4. **Prova ao vivo (live-probe)** dos endpoints de leitura relevantes, com a credencial real, para confirmar comportamento efetivo (status HTTP, exigência de parâmetros, formato de resposta).

3. Panorama das duas versões

(relatório anterior)			
Paths	60	68	4
Operações	60	86	5
Estilo	RPC (/v1/Módulo/Ação)	REST (/v2/recursos), paginado	REST, subconjunto mínimo
Módulos	14	17	3

v2 é uma reescrita RESTful, mais rica, com **paginação** (Page/PageSize) em vários recursos. **v3** é, hoje, um **subconjunto minúsculo** (só `PreCollaborator`, `CompanyUser` e `Request`) — **não traz nenhuma capacidade de descoberta**; para efeito da nossa pergunta, **a v3 é a mais pobre das três**.

Módulos por versão

- **v2 (17 módulos):** Dictionary(30), Vacancy(8), Datasource(6), Client(5), ClientRequest(5), Match(5), Request(5), RequestMatch(5), ClientHeadquarter(4), CompanyUser(3), KillerQuestion(3), CustomField(2), **Candidate(1)**, **PreCollaborator(1)**, RequestFolder(1), RequestUser(1), VacancyFolder(1).
- **v3 (3 módulos):** Request(3), CompanyUser(1), **PreCollaborator(1)**.

4. Resposta ao ponto central: o recurso `PreCollaborator`

Versão	Endpoints do módulo <code>PreCollaborator</code>	Listagem/Descoberta?	Evidência ao vivo
v1	GET /■v1/■PreCollaborator/■Get?■idPreCollaborator=■{id}	Não — só get-by-id	HTTP 404 (id=1)
v2	GET /■v2/■precollaborators/■{idPreCollaborator}	Não — só get-by-id	HTTP 404 (id=1)

v3	GET /v3/precollaborators/{idPreCollaborator}	Não — só get-by-id	HTTP 404 (id=1)
----	---	--------------------	-----------------

Complementarmente, a v2 tem o módulo **candidate**, mas com **um único endpoint de escrita**: POST /v2/candidates ("Cria um novo candidato na Base Própria"). **Não é listagem** — não ajuda no discovery.

Portanto, o endpoint literalmente pedido (PreCollaborator/List / /Search / changes-since) não existe em nenhuma das três versões. Se a decisão dependesse só disso, a conclusão da v1 se mantinha inalterada. **O que muda o jogo está no módulo Match da v2 (§5).**

5. A descoberta: enumeração de candidatos via Match (novidade da v2)

5.1 O endpoint novo

GET /v2/matches?IdVacancy={id}&IdVacancyFolder={id}&Page={n}&PageSize={n}

Na **v1**, Match só tinha Get?idMatch= (get-by-id). Na **v2**, passou a existir a **listagem paginada**, filtrável por **vaga** (IdVacancy) e por **etapa/pasta** (IdVacancyFolder).

5.2 O que cada item retorna (campos relevantes ao discovery)

Confirmado no schema MatchResponse **e ao vivo** (nomes de campo observados na resposta real):

Campo	Relevância
idMatch, idCandidate, idVacancy	Identificadores da inscrição/candidato/vaga
idVacancyFolder	Etapas atuais — permite mirar a pasta de admissão
cpf	Chave de identidade do EA (§A.3) — PII, nunca logada
name, surname, email, phone, birthDate, cep, address	Dados cadastrais do candidato
modifyDate, insertDate	Carimbos de tempo → viabilizam delta no cliente
hasBeenSentToERP, requests[], job	Estado/vaga

Ausência crítica: o MatchResponse **não contém idPreCollaborator**. Ver §6.1.

5.3 Caminho de descoberta proposto (e a evidência ao vivo de cada passo)

Passo	Chamada	Resultado ao vivo	O que prova
A	GET /v2/vacancies?Page&6821Size	HTTP 200, totalItems =	Enumera todas as vagas (paginado)
B	GET /v2/vacancy-folders?IdVacancy=847	HTTP 200, 6 etapas	Descobre as etapas da vaga (ver §5.4)
C	GET /v2/matches?IdVacancy=847&Page=1&PageSize=10	HTTP 200, totalItems = 253	Lista os candidatos da vaga, com CPF + etapa + datas

Encadeando **A → B → (achar a pasta de admissão) → C filtrando por IdVacancyFolder**, o cron-pull **consegue descobrir os candidatos que chegaram à etapa de admissão** — sem webhook. **Este é o achado.**

5.4 Etapas reais observadas (vaga de teste 847)

Lead · Inscritos · Pré-selecionado · Finalistas · **Contratados** · Descartados

A etapa de interesse para a admissão é a de contratação (aqui, **Contratados**). **Mas os nomes das pastas são livres por vaga** — não há um id/tipo canônico de "admissão"; exige de/para (§6.4).

5.5 Restrição de uso observada ao vivo (importante para quem for implementar)

Embora o swagger marque os parâmetros como *opcionais*, a API real exige **escopo + paginação**:

Chamada	HTTP
GET /v2/matches (sem parâmetros)	400
GET /v2/matches?IdVacancy=847 (sem Page/PageSize)	400
GET /v2/matches?IdVacancy=847&Page=1&PageSize=1	200

Ou seja: **não existe "listar todos os candidatos" global** — a listagem é **sempre por vaga**, e **Page/PageSize** são obrigatórios na prática. Isso reforça o problema de fan-out (§6.3).

6. Lacunas e riscos — por que isto ainda não substitui o webhook

6.1 O Match não leva ao PreCollaborator (nem aos documentos)

A listagem descobre o **candidato** (CPF, nome, vaga, etapa), mas: - **não retorna idPreCollaborator**, e - os **documentos** (que alimentam a auditoria F2, §A.4) só existem em GET /vN/precollaborators/{idPreCollaborator} → Documents[[]].

Não há, no catálogo, uma rota Match → PreCollaborator (nem precollaborators?idMatch=). **Sem confirmar como obter o idPreCollaborator a partir de um match, a descoberta via Match fica incompleta para o fluxo documental.** → Item nº 1 a alinhar com o Pandapé Operações.

6.2 Não há delta no servidor

Nenhum endpoint aceita `modifiedSince/updatedAfter`. O *delta* teria de ser feito **no cliente**, diffando `modifyDate/insertDate` contra o último ciclo. É **viável** (o EA já é idempotente por id, §A.5), mas significa **puxar páginas e filtrar localmente**, não pedir só "o que mudou".

6.3 Fan-out sob o teto de rate limit

São **6.821 vagas**. Uma varredura ingênua (para cada vaga: folders + páginas de matches) a cada tick do cron **estoura o teto de 1.000 req/5min compartilhado** — que é **requisito de segurança** (o excesso do EA pode atrasar o webhook G.Infor que alimenta a folha, §A.5). Um desenho viável exigiria **escopo agressivo**: só vagas ativas, cache dos `idVacancyFolder` de admissão, paginação apenas de registros recentes por `modifyDate`. **É um problema de engenharia real, não um detalhe.**

6.4 Identificação da pasta de admissão exige de/para

Como os nomes de etapa são livres por vaga (§5.4), decidir "qual pasta é a de admissão" precisa de um **de/para por nome/convenção** — mais um insumo a validar com o Pandapé e com a operação do RH.

7. Alinhamento com o Pandapé (registrado, não bloqueia esta análise)

O Pandapé solicitou que **qualquer novo endpoint a ser usado seja alinhado com o time de Operações deles antes de entrar em produção**. Registrado. Isso **não afeta a investigação** (concluída aqui), mas é **pré-condição de produção**. Pontos a levar a esse alinhamento:

1. Como obter o `idPreCollaborator` (e seus documentos) a partir de um match/candidato listado (§6.1).
2. Existe uma forma **canônica** de identificar a etapa de "admissão/contratado" (§6.4)?
3. Uso pretendido de GET /v2/matches como mecanismo de descoberta periódica é **suportado/permitido** pela política de uso deles, e qual a **cota** recomendada sob o rate limit compartilhado (§6.3)?
4. Existe (mesmo que não documentado no swagger) algum endpoint de **delta/"changes since"** ou de **notificação** que evite o fan-out?

8. Conclusão e recomendação

- Pergunta literal (PreCollaborator/List, /Search, changes-since):** não existe em v2 nem v3 — PreCollaborator é *get-by-id* nas três versões (v2/v3 confirmados ao vivo, HTTP 404).
- Novidade real da v2:** o módulo **Match** passou a oferecer **listagem paginada por vaga e etapa** (GET `/v2/matches`), com **CPF, etapa e carimbos de tempo** — **confirmado ao vivo (HTTP 200, 253 candidatos numa vaga)**. É uma **capacidade de descoberta de candidatos** que a v1 não tinha, e **reabre a discussão cron-pull × webhook**.
- Porém, não é troca automática de arquitetura.** Faltam quatro peças (§6): ligação `match→pré-colaborador/documentos`, *delta* server-side, viabilidade de fan-out sob rate limit, e de/para da etapa de admissão. **A descoberta é promissora, não conclusiva.**
- Recomendação técnica:** - **Autorizar uma prova de conceito (spike) de viabilidade** — sem produção — para: (a) medir o custo real de fan-out numa amostra de vagas ativas; (b) validar o *delta* por `modifyDate`; (c) descobrir experimentalmente a ligação `match→pré-colaborador`. - **Levar os quatro pontos da §7 ao time de Operações do Pandapé** antes de qualquer decisão de produção. - **Manter o webhook como opção viva** até o spike concluir: ele continua sendo o **único mecanismo de push** e a única via hoje comprovada de obter o `idPreCollaborator` de forma direta. - **Não implementar nada** até a decisão de arquitetura (conforme instrução).
- Veredito de uma linha:** a v3 não ajuda; a v2 abre uma porta real de *pull* para candidatos que antes não existia — o suficiente para **reavaliar seriamente** o cron-pull, **insuficiente** para declarar o webhook dispensável sem um spike e o aval do Pandapé Operações.

Anexo A — Evidências ao vivo (2026-07-01, credencial real)

#	Chamada	HTTP	Observação (sem PII)
1	POST <code>/connect/token</code> (OAuth)	200	Bearer emitido (não exibido)
2	GET <code>/v2/vacancies?Page=1&PageSize=2</code>	200	paginado; <code>totalItems = 6821</code>
3	GET <code>/v2/matches</code> (sem params)	400	listagem global rejeitada
4	GET <code>/v2/matches?IdVacancy=1847</code> (sem paginação)	400	<code>Page/PageSize</code> obrigatórios na prática
5	GET <code>/v2/matches?IdVacancy=1847&Page=1&PageSize=1</code>	200	<code>totalItems = 253</code> ; itens com <code>cpf, idVacancy, folder, modifyDate, insertDate</code> sem <code>idPreCollaborator</code>
6	GET <code>/v2/vacancy-folders?IdVacancy=1847</code>	200	6 etapas (Lead, Inscritos, Pré-selecionado, Finalizado)
7	GET <code>/v3/requests?Page=1&PageSize=1</code>	200	paginado
8	GET <code>/v2/precollaborators/1</code>	404	get-by-id, id inexistente
9	GET <code>/v3/precollaborators/1</code>	404	get-by-id, id inexistente

Anexo B — Catálogo v2 (68 paths / 86 operações), por módulo

Candidate — POST `/v2/candidates` (criar; escrita). **PreCollaborator** — GET `/v2/precollaborators/{idPreCollaborator}` (get-by-id). **Match** — GET `/v2/matches` (novo: *listagem por vaga/etapa*) · POST `/v2/matches` · GET `/v2/matches/{idMatch}` · GET `/v2/matches/{idMatch}/questionnaires` · PATCH `/v2/matches/{idMatch}/update`. **Vacancy** — GET `/v2/vacancies` (paginado) · POST `/v2/vacancies` · GET `/v2/vacancies/templates` · GET `/v2/vacancies/templates/{id}` · PUT `/v2/vacancies/updatevacancyusers` · GET `/v2/vacancies/{idRequest}` · PUT `/v2/vacancies/{idVacancy}` · PATCH `/v2/vacancies/{idVacancy}`. **VacancyFolder** — GET `/v2/vacancy-folders?IdVacancy=`. **Request** — POST `/v2/requests` · GET `/v2/requests?IdVacancy=` · GET `/v2/requests/{idRequest}` · PATCH `/v2/requests/{idRequest}` ·

PATCH /v2/requests/{idRequest}/assign-vacancy. **RequestFolder** — GET /v2/request-folders/{idRequest}. **RequestMatch** — POST /v2/request-matches · GET /v2/request-matches?idMatch= · PATCH /v2/request-matches/{id} · GET /v2/request-matches/{id} · GET /v2/request-matches/{id}/evaluations. **RequestUser** — GET /v2/requests/{idRequest}/users. **Client** — GET /v2/clients (paginado) · POST · DELETE · GET /v2/clients/{idClient} · PUT /v2/clients/{idClient}. **ClientHeadquarter** — GET /v2/clients/headquarters (paginado) · POST · GET .../{id} · PATCH .../{id}. **ClientRequest** — POST .../{idHeadquarter}/requests · PUT/DELETE .../{idRequest} · GET /v2/clients/requests?idVacancy= · GET /v2/clients/requests/{idRequest}. **CompanyUser** — POST /v2/company/users · GET /v2/company/users (paginado, Role) · GET .../{id}. **CustomField** — GET /v2/custom-fields · GET /v2/custom-fields/dictionary-values/{id}. **Datasource** — GET /v2/data-sources (paginado) · POST · GET .../items (paginado) · GET .../items/{id} · POST .../{id}/items · DELETE .../{id}/items. **KillerQuestion** — GET/POST/DELETE /v2/killer-questions/{idVacancy}. **Dictionary (30)** — tabelas de domínio (categorias, localidades, idiomas, escolaridade, contrato, raça, gênero, etc.); nenhuma relacionada a descoberta de candidatos.

Anexo C — Catálogo v3 (4 paths / 5 operações)

- PreCollaborator** — GET /v3/precollaborators/{idPreCollaborator} (get-by-id).
- Request** — GET /v3/requests?idVacancy&Page&PageSize · PATCH /v3/requests/{id} · PUT /v3/requests/{id}.
- CompanyUser** — POST /v3/company/users.

Nenhum endpoint de listagem/descoberta de pré-colaboradores ou candidatos na v3.

Anexo D — Referências verificáveis

Fonte	Referência
Spec v2	https://api.pandape.com.br/swagger/v2/swagger.json (OpenAPI 3.0.1, "Pandapé API v2", 68 paths)
Spec v3	https://api.pandape.com.br/swagger/v3/swagger.json (OpenAPI 3.0.1, "Pandapé API v3", 4 paths)
UI v2	https://api.pandape.com.br/index.html?urls.primaryName
UI v3	https://api.pandape.com.br/index.html?urls.primaryName
Relatório da v1	docs/RELATORIO-INVESTIGACAO-API-PANDAPE.md
Código atual (cliente v1)	apps/backend/src/pandape/pandape-api.service.ts

Documento gerado para subsídio à decisão de arquitetura. Não contém dados pessoais de candidato. Nenhuma alteração de código foi realizada — investigação apenas.